

BASIL LEAVES DISEASE DETECTION

A report submitted in partial fulfillment of the requirements for the Degree of

Bachelor of Technology

in

COMPUTER SCIENCE & ENGINEERING

by

N. Nihar

2111CS010315

Shyam.B

2111CS010304

D. Padma Srija

2111CS010347

CH. Nikhilesh Varma

2111CS010325

Under the esteemed guidance of

Dr. Arun Singh Chouhan

(Associate Professor)



Department of Computer Science & Engineering

School of Engineering

MALLA REDDY UNIVERSITY

Maisammaguda, Dulapally, Hyderabad, Telangana 500100

2024-2025

BASIL LEAVES DISEASE DETECTION

A report submitted in partial fulfillment of the requirements for the Degree of

Bachelor of Technology

in

COMPUTER SCIENCE & ENGINEERING

by

N. Nihar 2111CS010315

Shyam.B 2111CS010304

D.Padma Srija 2111CS010347

CH.Nikhilesh Varma 2111CS010325

Under the esteemed guidance of

Dr. Arun Singh Chouhan

(Associate Professor)



**Department of Computer Science & Engineering School of
Engineering**

MALLA REDDY UNIVERSITY

Maisammaguda, Dulapally, Hyderabad, Telangana 500100

2024-2025



MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

Department of Computer Science & Engineering

CERTIFICATE

This is to certify that the project report entitled “**BASIL LEAVES DISEASE DETECTION**”, submitted by NANDALANIHAR (2111CS010315), Shyam. B (2111CS010304), D. PADMA SRIJAI (2111CS010347), CH. NIKHILESH VARMA (2111CS010325), towards the partial fulfillment for the award of Bachelor’s Degree in *Bachelor of Technology (B. Tech)* from the Department of Computer Science & Engineering, Malla Reddy University, Hyderabad, is a record of bonafide work done by them. The results embodied in the work are not submitted to any other University or Institute for award of any degree or diploma.

Internal Guide
Dr. Arun Singh Chouhan

Dean of the Department
Dr. Shaik Meeraval

External Examiner

DECLARATION

We hereby declare that the project report entitled “**Basil Leaves Disease Detection**” has been carried out by us and this work has been submitted to the Department of **Computer Science & Engineering**, Malla Reddy University, Hyderabad in partial fulfillment of the requirements for the award of degree of Bachelor of Technology. We further declare that this project work has not been submitted in full or part for the award of any other degree in any other educational institutions.

Place: Hyderabad

Date:

N. Nihar	2111CS010315
Shyam. B	2111CS010304
D.Padma Srija	2111CS010347
CH.Nikhilesh Varma	2111CS010325

ACKNOWLEDGEMENT

We extend our sincere gratitude to all those who have contributed to the completion of this project report. Firstly, we would like to extend our gratitude to **Dr. V. S. K Reddy**, Vice Chancellor, for his visionary leadership and unwavering commitment to academic excellence

We would also like to express my deepest appreciation to our project guide **Dr. Arun Singh Chouhan** Associate Professor, whose invaluable guidance, insightful feedback, and unwavering support have been instrumental throughout the course of this project for successful outcomes.

We are also grateful to **Dr. Shaik Meeravali**, Dean of the Department of Computer Science And Engineering, for providing us with the necessary resources and facilities to carry out this project.

We are deeply indebted to all of them for their support, encouragement, and guidance, without which this project would not have been possible.

N. Nihar	(2111CS010315)
Shyam. B	(2111CS010304)
D.PadmaSrija	(2111CS010347)
CH.Nikhilesh Varma	(2111CS010325)

ABSTRACT

The agricultural sector plays a crucial role in sustaining human life, and plant health directly impacts crop yield and food security. In this project, we present a deep learning-based approach for Basil plant disease detection using Convolutional Neural Networks (CNNs). Our goal is to assist farmers and gardeners in identifying and managing plant diseases at an early stage with higher accuracy and minimal manual intervention.

A dataset containing basil leaf images classified into four categories—Bacterial, Fungal, Pest, and Healthy—was used to train the model. The CNN model was developed and trained using TensorFlow and Keras, achieving reliable accuracy in classifying the diseases based on visual symptoms. A user-friendly web application was built using Flask, allowing users to upload images of basil leaves and receive instant predictions along with suggested remedies for detected diseases.

This intelligent system significantly reduces the time, cost, and dependency on expert agricultural advice by providing automated disease detection. While the model performs efficiently, challenges such as limited dataset diversity, image quality, and generalization across different plant species remain. Future improvements can focus on expanding the dataset, integrating real-time leaf monitoring via mobile apps, and supporting multiple plant types.

This project demonstrates how AI and Deep Learning can empower sustainable agriculture by providing accessible, scalable, and effective plant disease management solutions

CONTENTS

DESCRIPTION	PAGE NUMBER
CERTIFICATE	ii
DECLARATION	iii
ACKNOWLEDGEMENTS	iv
ABSTRACT	v
LIST OF FIGURES	viii
CHAPTER 1 INTRODUCTION	1
1.1 Problem Statement	1-2
1.2 Objectives	2-3
1.3 Goal of the project	3
CHAPTER 2 LITERATURE SURVEY	4
CHAPTER 3 SYSTEM ANALYSIS & DESIGN	5-8
3.1 Existing System	5
3.2 Proposed System	5
3.3 Design	6-8
3.3.1 System Architecture	6
CHAPTER 4 SYSTEM REQUIREMENTS & SPECIFICATIONS	9-11
4.1 Dataset	9
4.2 System Requirements	10-11
4.2.1 Software Requirements	10
4.2.2 Hardware Requirements	10
4.3 Functional Specifications	16-17
CHAPTER 5: DESIGN AND IMPLEMENTATION	18-52
5.1 System Architecture	
5.2 System Architecture Diagram	
5.3 UML Diagrams	
5.4 Data Flow Diagram (DFD)	
5.5 Sequence Diagram	
CHAPTER 6: SOURCE CODE AND SCREENSHOTS	53-58

CHAPTER 7 CONCLUSION & FUTURE ENHANCEMENT	59-60
7.1 Conclusion	59
7.2 Future Enhancement	59
REFERENCES	60

LIST OF FIGURES

FIGURE	TITLE	PAGE NUMBER
3.3.1	SYSTEM ARCHITECTURE	12
4.1.1	Non Hemorrhage Images Dataset	14
4.1.2	Hemorrhage Images Dataset	15
6.1	CNN Model Summary	53
6.2	Training Accuracy – Loss	54
6.3	Validaion Accuracy – Loss	54
6.4	Category Distribution	55
6.5	Feature Spread Visualization	55
6.6	Feature Frequency Analysis	56
6.7	Confusion Matrix	56
6.8	Login Interface	57
6.9	Patient Information Form	57
6.10	Diagnosis Result Interface	58

CHAPTER – 1

INTRODUCTION

Agriculture is the backbone of many economies, especially in developing countries, and plant health plays a pivotal role in ensuring food security and sustainable farming practices. However, early and accurate detection of plant diseases remains a significant challenge for farmers due to limited access to expert agricultural guidance and the often subtle visual symptoms of disease. Traditional disease identification methods are manual, time-consuming, and prone to human error, which can lead to delayed treatments and reduced crop yields.

With advancements in Artificial Intelligence (AI), especially in the field of Deep Learning, it is now possible to automate the process of disease detection with high precision. This project—Basil Plant Disease Detection using Deep Learning—aims to develop a smart, AI-powered system that can classify basil plant leaf images into four categories: Bacterial, Fungal, Pest, and Healthy. By leveraging Convolutional Neural Networks (CNNs), the system can accurately detect visible symptoms on the leaves and suggest appropriate remedies to mitigate the problem.

The system is implemented using Python, TensorFlow, and Keras for the deep learning model, and a simple web-based interface built using Flask enables real-time image uploads and instant predictions. Users can upload images of basil leaves through the interface, after which the model processes the image and outputs the disease type along with a recommended solution.

This intelligent plant care assistant not only minimizes dependency on expert intervention but also enhances productivity by enabling timely and accurate diagnosis. With further refinement, the model has the potential to support detection in a wider range of crops and be integrated into mobile platforms for real-world deployment in agriculture.

1.1 Problem Statement

In the agricultural sector, plant diseases remain a critical challenge, significantly affecting crop yield, quality, and farmer income. Among various crops, basil—an essential herb widely used in culinary and medicinal applications—is particularly susceptible to diseases such as bacterial infections, fungal growth, and pest attacks. These diseases not only diminish plant health but can also spread rapidly if not

detected and treated early.

Traditional disease detection methods rely heavily on manual observation by farmers or agricultural experts, which is often time-consuming, subjective, and prone to human error. In rural and resource-constrained areas, access to plant pathologists or extension services may be limited or delayed, resulting in late intervention and increased crop loss. Additionally, early symptoms of many diseases are visually similar, making accurate diagnosis even more difficult without expert guidance.

The lack of affordable, accessible, and automated disease detection tools contributes to widespread misdiagnosis and the misuse of pesticides or fungicides, which can further degrade plant health and harm the environment. There is a pressing need for a reliable and user-friendly solution that enables farmers and gardeners to quickly identify the health status of their basil plants and receive appropriate remedial suggestions.

To address this challenge, this project proposes a deep learning-based web application capable of classifying basil leaf images into four categories—Bacterial, Fungal, Pest, and Healthy. This AI-powered system aims to democratize access to plant disease diagnostics, reduce reliance on manual inspection, and support farmers in taking timely, accurate, and sustainable action to protect their crops.

1.2 OBJECTIVES

- Develop an AI model using Convolutional Neural Networks (CNN) to accurately detect diseases in basil leaves.
- Classify basil leaf images into categories like Bacterial Spot, Leaf Blight, and Healthy.
- Create a user-friendly web interface for farmers and researchers to upload leaf images and get instant disease predictions.
- Use image preprocessing techniques to improve the model's prediction accuracy and robustness.

- Train and validate the model on a structured dataset with balanced disease categories.
- Deploy the solution using Flask to make the model accessible through a web browser.
- Help farmers take timely action by identifying diseases early, reducing crop loss, and improving yield quality.

1.3 GOAL OF THE PROJECT

The primary goal of this project is to develop an AI-powered web application that enables accurate, fast, and user-friendly identification of diseases in basil plants. By utilizing deep learning—specifically Convolutional Neural Networks (CNNs)—the system aims to classify basil leaf conditions into four categories: Bacterial, Fungal, Pest, and Healthy with high precision

The project is focused on providing an accessible platform where users, especially farmers and plant enthusiasts, can upload an image of a basil leaf and instantly receive a diagnosis along with a recommended remedy. This minimizes dependency on expert agricultural advice, speeds up disease detection, and helps users take corrective measures before diseases spread.

Another key goal is to promote smart and sustainable farming by providing a low-cost and scalable solution for early plant disease diagnosis. This will help reduce yield loss, improve crop quality, and empower users with AI-driven decision-making support.

The system also aims to offer a seamless user experience through a web-based interface developed using modern frontend and backend technologies. Security, speed, and ease of use are central to this goal to ensure widespread adoption and usability.

Additionally, the project strives to protect data privacy while delivering a reliable and scientifically accurate diagnosis system. By achieving these goals, the Basil Plant Disease Detection System stands as an innovative step toward integrating artificial intelligence into agriculture for better plant health management and productivity.

CHAPTER – 2

LITERATURE SURVEY

The use of deep learning and image classification for plant disease detection has been widely researched in recent years due to increasing global demand for sustainable agriculture and crop protection. Early approaches to plant disease detection relied heavily on manual observation and traditional machine learning techniques, which required handcrafted features and had limited accuracy. However, with the rise of deep learning, especially Convolutional Neural Networks (CNNs), automated and high-accuracy disease classification has become feasible.

Mohanty et al. demonstrated the effectiveness of deep CNNs in identifying 26 different diseases across 14 crop species using a large dataset of leaf images. Their research proved that deep learning models can outperform traditional approaches and eliminate the need for manual feature extraction. Similarly, Sladojevic et al. developed a CNN-based model for real-time plant disease recognition, achieving promising accuracy across multiple crop types.

Zhang et al. focused on fine-grained disease classification in leaves using an enhanced CNN model, introducing data augmentation and transfer learning to improve the robustness of predictions. Their approach is particularly relevant for this project, where different types of basil diseases (fungal, bacterial, pest damage) may exhibit visually subtle differences.

Transfer learning has been a major advancement in this domain. Ferentinos employed pre-trained models like VGG16 and AlexNet, which drastically reduced training time while maintaining high accuracy, especially on smaller datasets. This approach is valuable when collecting a vast number of labeled plant disease images is difficult.

Challenges still exist in distinguishing between visually similar disease patterns or handling real-world scenarios with noise, varied lighting, or multiple diseases on the same leaf. To address this, Abbas et al. proposed ensemble models and hybrid approaches combining CNNs with support vector machines (SVMs), which improved reliability in complex conditions.

Studies also stress the importance of proper dataset preparation, preprocessing, and augmentation techniques to avoid overfitting and to ensure that the model generalizes well across different environmental conditions. Techniques such as rotation, flipping, and scaling are commonly used to simulate real-world variability in plant leaves.

CHAPTER – 3

SYSTEM ANALYSIS & DESIGN

3.1 EXISTING SYTEM

In the current agricultural landscape, the detection of basil leaf diseases primarily relies on manual observation by farmers or agricultural experts. While this method has been practiced for years, it comes with major drawbacks. Manual inspection is time-consuming, especially when early detection is crucial to prevent crop loss. It also requires the presence of trained professionals, which may not always be available in rural or under-resourced farming regions. Moreover, the accuracy of detection can vary due to human error, fatigue, and lack of experience. Although some AI-based tools have been developed in the past, many of them use basic machine learning models without user-friendly interfaces, making them hard for farmers to use. These tools often lack real-time prediction capability and visual interpretability, limiting their practical use in real farming environments.

3.2 PROPOSED SYSTEM

The proposed system is developed to overcome the limitations of traditional manual leaf disease detection by providing an AI-powered, real-time, and user-friendly web solution. It uses a Convolutional Neural Network (CNN) trained on a dataset of basil leaf images to accurately classify whether a leaf is healthy or affected by diseases such as Downy Mildew, Bacterial Leaf Spot, etc.

The system is integrated into a full-stack web application with an HTML/CSS/JavaScript frontend and a Flask backend. When a user uploads a basil leaf image, the system preprocesses it (resizing, normalization) and passes it through the trained CNN model hosted via the Flask API. The frontend displays the prediction result—detected disease name or “Healthy”—along with a confidence score.

To enhance usability, the system also includes an optional SQLite database to store image records and prediction history for future reference. This solution improves accuracy, reduces the need for expert intervention, and supports farmers and agricultural workers in making faster, more informed decisions in the field.

3.3 DESIGN

3.3.1 SYSTEM ARCHITECTURE

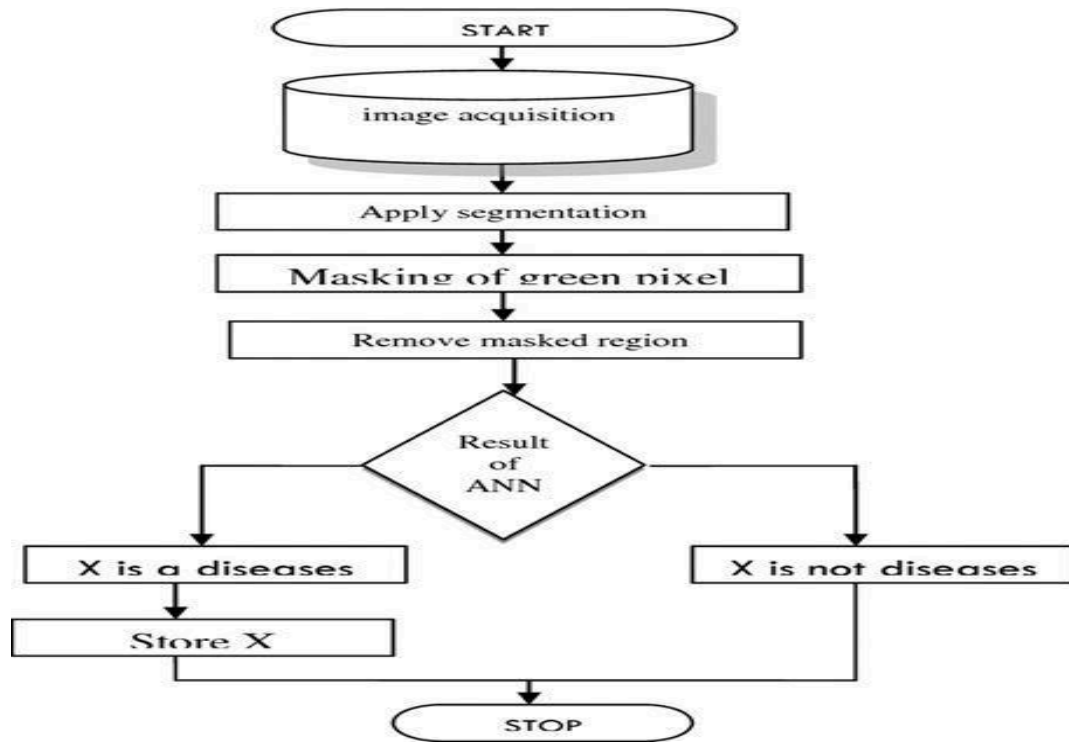


Fig 3.3.1 Proposed System Architecture

n Figure 3.3.1, the architecture of the Basil Leaf Disease Detection system using Convolutional Neural Networks (CNN) is depicted. The process begins with the basil leaf images undergoing preprocessing steps, including resizing, normalization, and augmentation. These processed images are then passed through the CNN layers for feature extraction and classification, ultimately predicting whether the leaf is healthy or affected by diseases like Downy Mildew or Bacterial Leaf Spot.

1. Image Acquisition

The first and foundational step in the system architecture is Image Acquisition. This stage involves collecting high-quality images of basil leaves under various conditions (healthy and diseased). These images serve as the primary input for the entire system.

Input Source: Images are captured using a smartphone camera, DSLR, or a dedicated imaging device.

Image Format: JPEG, PNG, or any standard image format.

Environment Considerations: Consistent lighting, neutral background, and high resolution are maintained to enhance detection accuracy.

Preprocessing Needs: Images may be resized, normalized, or augmented to prepare them for further processing.

This step ensures the availability of a robust and diverse dataset, which is critical for training and testing the disease detection model.

2.Data Preprocessing

Before feeding the images into the CNN model, they undergo preprocessing, which includes:

- **Resize:** Images are resized to a standard dimension for consistency.
- **Normalize:** Pixel values are normalized to improve model efficiency.
- **Augmentation:** Data augmentation techniques like flipping, rotation, and zooming are applied to enhance model generalization and handle limited data.

3.CNN Architecture

The core of the system is the **Convolutional Neural Network (CNN)**, which consists of several layers:

a. Convolutional Layers

- Feature extraction is done using multiple filters (e.g., 128 filters of size 3x3).
- **ReLU activation** and **padding** are applied to introduce non-linearity and preserve image dimensions.

b. Pooling Layers

- Max pooling operations reduce the spatial dimensions while retaining important features.
- Multiple layers of pooling improve robustness to spatial variations.

c. Fully Connected Layers

- Flattened features from the convolutional layers are passed to dense layers to learn complex relationships.
- These layers are responsible for classification.

d. Dropout Layers

- Dropout is used at multiple points to prevent overfitting by randomly dropping neurons during training.

e. Activation Functions

- **ReLU** and **Softmax** functions are applied to introduce non-linearity and perform final classification respectively.

2. Output Layer

The final classification is done using a **Softmax** layer which outputs probabilities for:

- **Fungus**
- **Healthy**
- **Pest**
- **Bacterial**

The class with the highest probability is considered the model's prediction.

CHAPTER – 4

SYSTEM REQUIREMENTS & SPECIFICATIONS

2.1 DATASET



Fig 4.1.1 Images Dataset



Figure 4.1.1, a sample of the dataset containing **healthy basil leaf images** is presented. These images showcase basil leaves without any signs of disease and are used to help the model learn the patterns of healthy leaf structures. This dataset is essential for training the model to accurately differentiate between healthy and diseased basil leave

2.2 SYSTEM REQUIREMENTS

2.2.1 SOFTWARE REQUIREMENTS

Operating System: Windows, macOS, or Linux.

Python Version: 3.7 or higher.

Libraries: TensorFlow , Keras , NumPy , Matplotlib , OpenCV , scikit-learn

IDE/Editor: VSCode or PyCharm or Jupiter Notebook.

CUDA/cuDNN (for GPU usage): Optional for faster training with an NVIDIA GPU.

2.2.2 HARDWARE REQUIREMENTS

Processor (CPU): Intel i5/i7 or AMD Ryzen 5/7.

Memory (RAM): 16 GB or more.

Graphics Card (GPU): NVIDIA GPU (e.g., GTX 1660 or higher) for faster computation (optional).

Storage: SSD with at least 100 GB of free space.

Network: Stable internet connection for downloading datasets and dependencies.

2.3 FUNCTIONAL SPECIFICATIONS

1. Login Functionality

The system begins with a secure login process. Users (e.g., farmers, agricultural specialists) must enter their credentials, such as a username and password, to gain access to the system. This ensures that only authorized users can upload images of basil leaves and view the results. If the user forgets their password, a password recovery option is available. The login page features a user-friendly interface with form validations and clear instructions for a smooth login experience. Once logged in, users are redirected to the main dashboard to upload images and access other features.

2. Image Upload

After logging in, users can upload basil leaf images using an easy-to-navigate React-based interface. The system includes an upload button and a drag-and-drop area for file selection. Supported image formats (e.g., PNG, JPEG) are accepted, and the system validates the uploaded image for format and size. After successful validation, a preview of the image is shown to confirm that the correct file has been uploaded. Once the image is confirmed, it is sent to the backend for further processing.

3. Prediction

Once the image is uploaded, it undergoes preprocessing, including resizing, normalization, and augmentation. The preprocessed image is then passed to a Convolutional Neural Network (CNN) model that analyzes the leaf's condition. The model classifies the leaf as either "Healthy" or "Diseased" based on learned patterns and provides a confidence score that indicates the accuracy of the prediction. The system ensures efficient processing, so the result is generated in real-time, minimizing user wait times.

4. Result Display

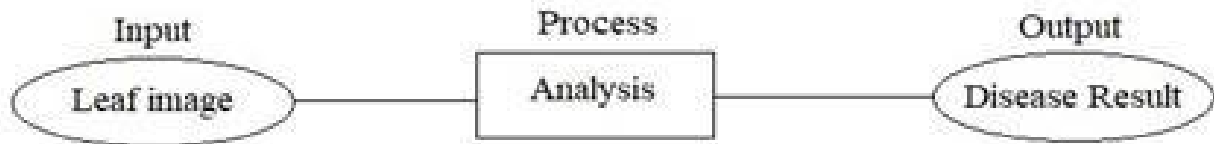
After the prediction is made, the results are displayed clearly on the frontend. The system shows whether the uploaded basil leaf image is classified as "Healthy" or "Diseased," along with the confidence score. The results are visually presented with color-coded feedback, such as green for "Healthy" and red for "Diseased." This makes it easier for users to interpret the results at a glance, enabling quick decision-making for further action like treatment or monitoring of the plant's health

CHAPTER – 5

DESIGN AND IMPLEMENTATION

5.1 System Architecture

The **Basil Leaves Disease Detection System** is designed to accurately identify diseases in basil plants through an automated and intelligent platform. The architecture consists of several integrated layers that work together to process leaf images, perform classification, and deliver real-time results. These layers include the frontend, backend, machine learning model, image processing components, and a database.



5.2 System Architecture Diagram

1. Input Layer

Image Acquisition Module

Accepts basil leaf images from the user (camera or file upload).

2. Preprocessing Layer

Segmentation Module

Applies image segmentation techniques (e.g., K-means, thresholding) to isolate leaf regions.

Green Pixel Masking

Masks out healthy green areas of the leaf.

Masked Region Removal

Removes the masked (non-disease) regions to focus only on potential diseased parts.

3. Classification Layer

Artificial Neural Network (CNN) Classifier

Inputs: Processed image data (disease-affected regions).

Outputs: Prediction result — "Disease" or "No Disease".

4. Decision Layer

If the result = “Disease”

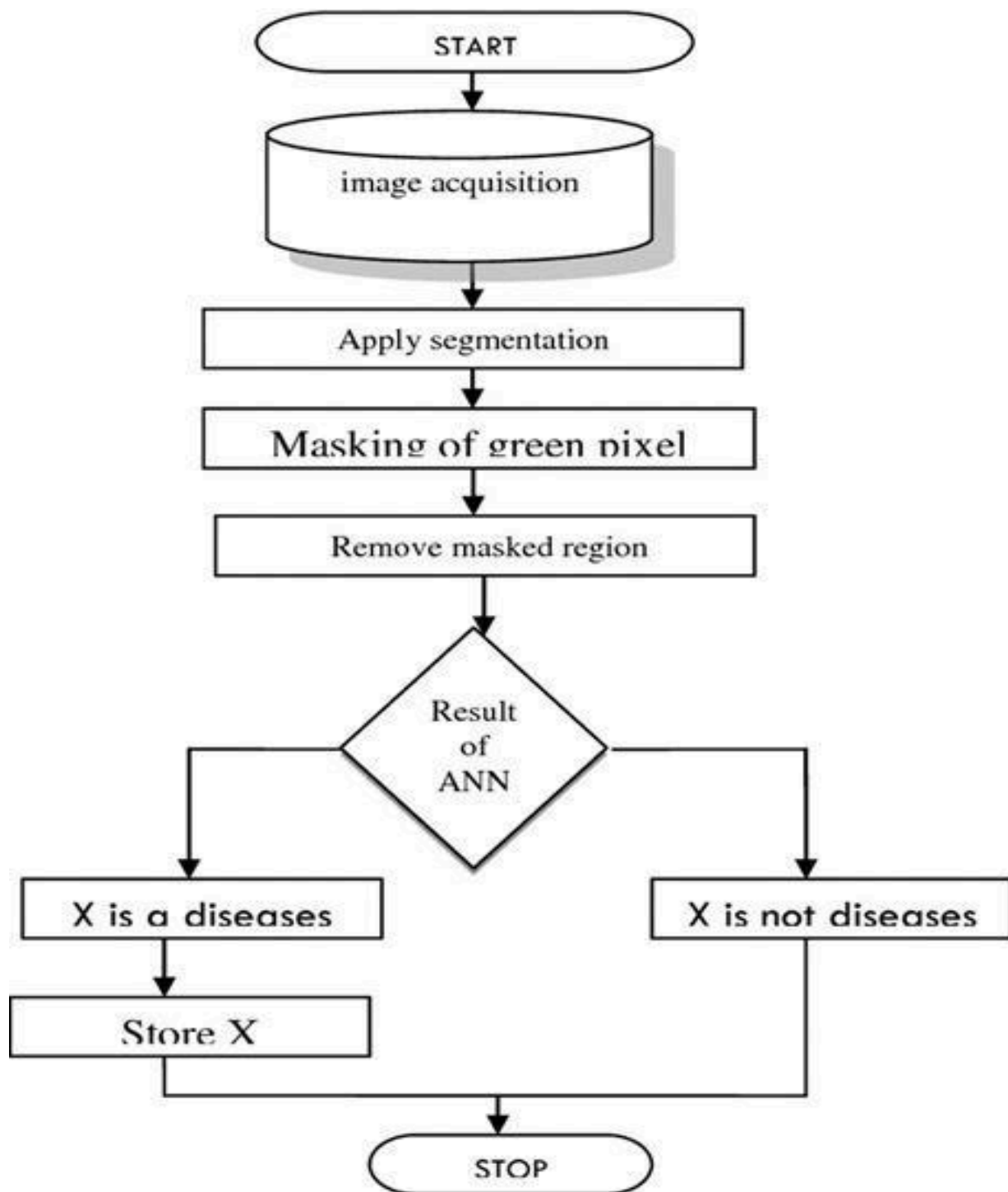
Store X in the database (record of infected leaf)

Else No action or store as healthy

5. Storage Layer

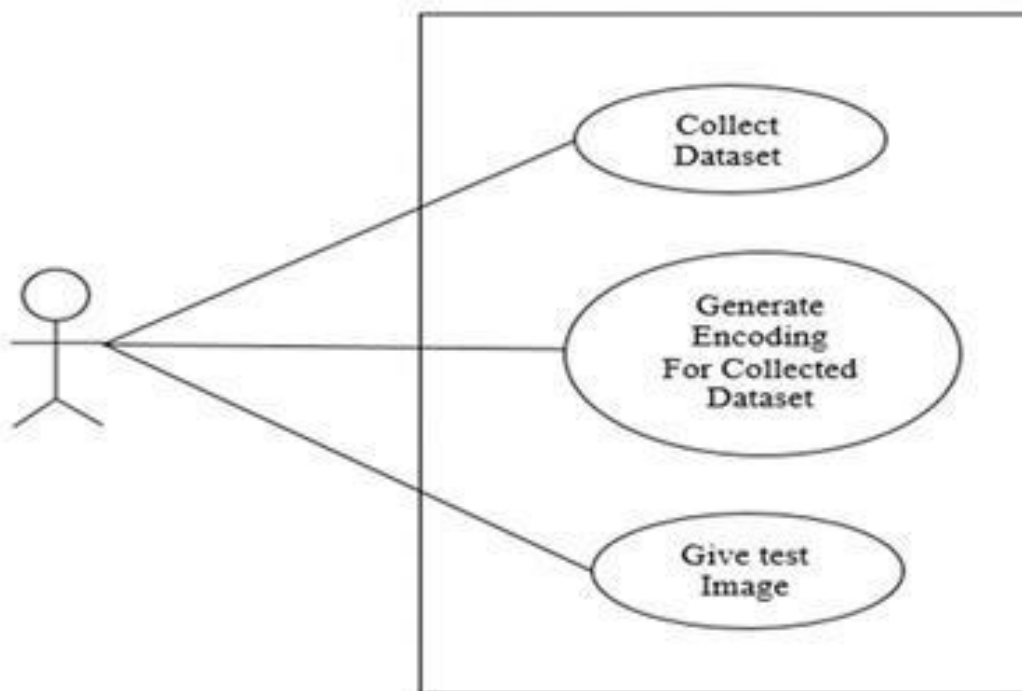
Result Storage Module

Stores image results, disease category, and timestamp in a database for reporting and analysis.



5.3 UML Diagrams

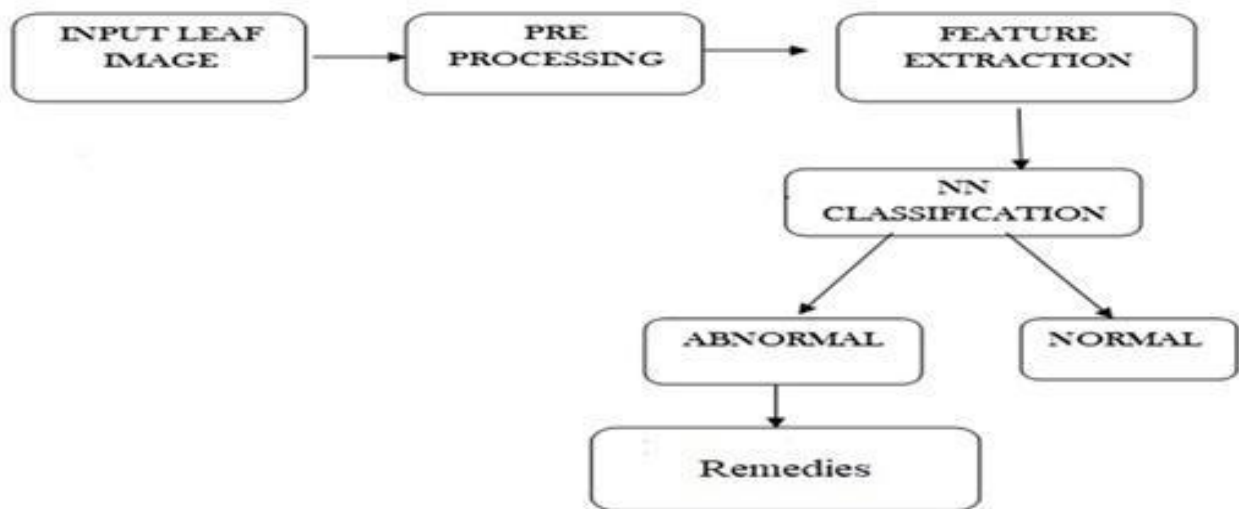
UML (Unified Modelling Language) diagrams help illustrate the structure, behaviour, and flow of the Basil Leaves Disease Detection System. These diagrams ensure a clear understanding of how users interact with the system and how data moves internally.



5.4 Data Flow Diagram (DFD)

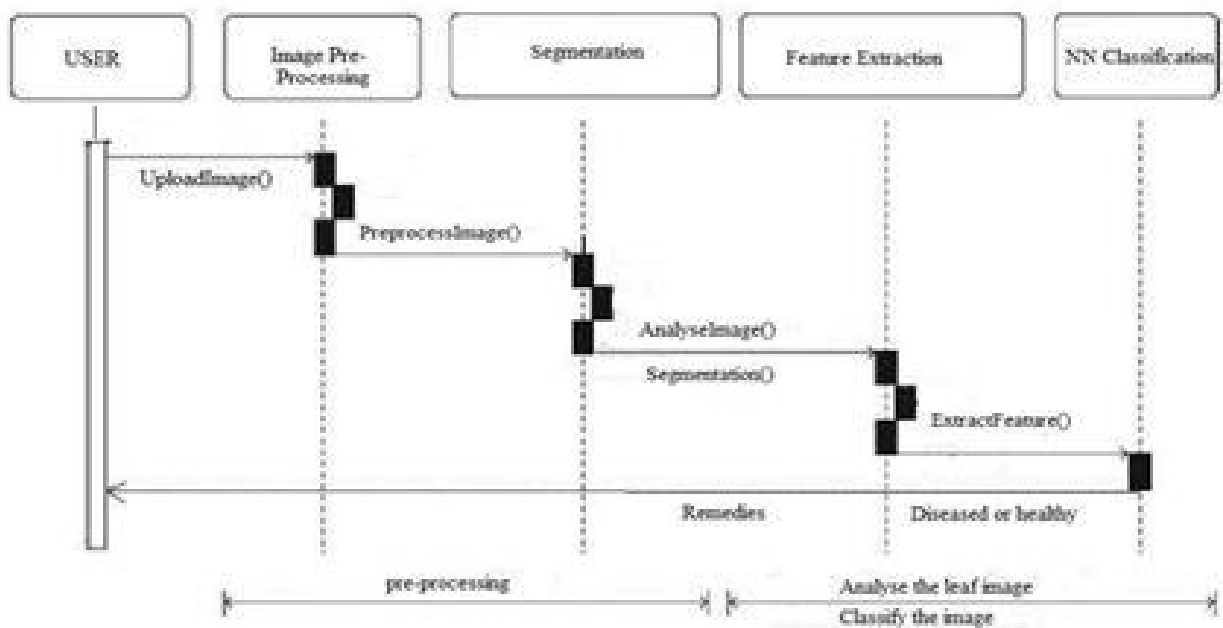
The **Data Flow Diagram (DFD)** outlines the flow of data within the system, from image upload to disease prediction and result storage.

- The user (farmer/admin) logs in and uploads a basil leaf image.
- The system preprocesses the image (resizing, cleaning, etc.).
- The pre-processed image is sent to the ML model for classification.
- The model returns a prediction (e.g., "Downy Mildew").
- The result is displayed on the frontend interface.
- The prediction and associated image data are stored in the database.
- The admin can view system usage, analyze common diseases, and manage stored records.



5.5 Sequence Diagram

A **sequence diagram** shows how the components of your system interact over time, from user input to disease detection and result display.



User → Web Interface: Uploads basil leaf image.

Web Interface → Image Processing Module: Sends image for preprocessing (resize, normalize).

Image Processing Module → Segmentation & Masking Module

Segmentation & Masking Module → ANN Classifier: Passes processed image.

ANN Classifier → Result Handler: Returns prediction result (Disease or Healthy).

Result Handler → Web Interface: Displays result to user.

Result Handler → Database: Stores result, image data, and prediction (only if diseased).

CHAPTER – 6

SOURCE CODE

Train_model.ipyb :

```
import os
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import classification_report, confusion_matrix
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout,
BatchNormalization
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, ReduceLROnPlateau,
CSVLogger
from tensorflow.keras.utils import plot_model

# Define paths
train_dir = 'dataset/train'
val_dir = 'dataset/val'
model_dir = 'model'
log_dir = 'logs'

os.makedirs(model_dir, exist_ok=True)
os.makedirs(log_dir, exist_ok=True)

# Setup data generators with augmentation
train_datagen = ImageDataGenerator(
    rescale=1./255,
    zoom_range=0.2,
    shear_range=0.2,
    rotation_range=30,
    width_shift_range=0.2,
    height_shift_range=0.2,
    horizontal_flip=True
```

```
)
```

```
val_datagen = ImageDataGenerator(rescale=1./255)
```

```
train_data = train_datagen.flow_from_directory(  
    train_dir,  
    target_size=(128, 128),  
    batch_size=32,  
    class_mode='categorical',  
    shuffle=True  
)
```

```
val_data = val_datagen.flow_from_directory(  
    val_dir,  
    target_size=(128, 128),  
    batch_size=32,  
    class_mode='categorical',  
    shuffle=False  
)
```

```
num_classes = train_data.num_classes  
class_labels = list(train_data.class_indices.keys())
```

```
# Build the CNN model
```

```
model = Sequential([  
    Conv2D(32, (3, 3), activation='relu', input_shape=(128, 128, 3)),  
    BatchNormalization(),  
    MaxPooling2D(),  
    Dropout(0.25),  
  
    Conv2D(64, (3, 3), activation='relu'),  
    BatchNormalization(),  
    MaxPooling2D(),  
    Dropout(0.3),
```

```

Conv2D(128, (3, 3), activation='relu'),
BatchNormalization(),
MaxPooling2D(),
Dropout(0.4),

Flatten(),
Dense(256, activation='relu'),
Dropout(0.5),
Dense(num_classes, activation='softmax')
])

model.summary()

# Save model structure image and summary
plot_model(model, to_file=os.path.join(model_dir, 'model_structure.png'), show_shapes=True,
show_layer_names=True)
with open(os.path.join(model_dir, 'model_summary.txt'), 'w') as f:
    model.summary(print_fn=lambda x: f.write(x + '\n'))

# Compile
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Callbacks
checkpoint = ModelCheckpoint(
    filepath=os.path.join(model_dir, 'best_model.h5'),
    monitor='val_accuracy',
    save_best_only=True,
    mode='max',
    verbose=1
)

early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', patience=3, factor=0.2, verbose=1)

```

```

csv_logger = CSVLogger(os.path.join(log_dir, 'training_log.csv'), append=False)

# Train model
history = model.fit(
    train_data,
    validation_data=val_data,
    epochs=30,
    callbacks=[checkpoint, early_stop, reduce_lr, csv_logger]
)

# Save final model
model.save(os.path.join(model_dir, 'final_model.h5'))

# Evaluate and save classification report
val_data.reset()
Y_pred = model.predict(val_data)
y_pred = np.argmax(Y_pred, axis=1)

conf_mat = confusion_matrix(val_data.classes, y_pred)
report = classification_report(val_data.classes, y_pred, target_names=class_labels)

with open(os.path.join(log_dir, 'classification_report.txt'), 'w') as f:
    f.write(report)

# Confusion Matrix Plot
plt.figure(figsize=(8, 6))
sns.heatmap(conf_mat, annot=True, fmt='d', cmap='Blues', xticklabels=class_labels,
            yticklabels=class_labels)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.savefig(os.path.join(log_dir, 'confusion_matrix.png'))
plt.close()

```

```

# Accuracy Plot
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.title('Accuracy over Epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid()
plt.savefig(os.path.join(log_dir, 'accuracy_plot.png'))
plt.close()

```

```

# Loss Plot
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Loss over Epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid()
plt.savefig(os.path.join(log_dir, 'loss_plot.png'))
plt.close()
print("Training complete. All files saved.")

```

App.py :

```

from flask import Flask, request, jsonify, render_template, redirect, url_for, flash
from werkzeug.utils import secure_filename
import os
import numpy as np
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing.image import load_img, img_to_array
import datetime
import logging

# Initialize Flask App

```

```

app = Flask(__name__)
app.secret_key = 'your_secret_key'

# Configure Upload Folder
UPLOAD_FOLDER = 'static/uploads'
ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg'}

app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
app.config['MAX_CONTENT_LENGTH'] = 16 * 1024 * 1024 # 16MB max file size

# Load Model
model = load_model('model/final_model.h5')

# Class Labels - Customize based on your dataset
class_names = ['Healthy', 'Bacterial Spot', 'Leaf Blight']

# Logging Setup
logging.basicConfig(filename='app_logs.log', level=logging.INFO)

# Helper Function to check allowed file
def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

# Home Route
@app.route('/')
def index():
    return render_template('index.html')

# Upload Page
@app.route('/upload', methods=['GET', 'POST'])
def upload_file():
    if request.method == 'POST':
        if 'file' not in request.files:
            flash("No file part")

```

```

        return redirect(request.url)

file = request.files['file']

if file.filename == "":
    flash("No file selected")
    return redirect(request.url)

if file and allowed_file(file.filename):
    filename = secure_filename(file.filename)
    filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
    file.save(filepath)

    logging.info(f'{datetime.datetime.now()} - File uploaded: {filename}')

    prediction, confidence = predict_disease(filepath)

    logging.info(f'{datetime.datetime.now()} - Prediction: {prediction} ( {confidence}%)')

    return render_template('result.html', filename=filename, prediction=prediction,
confidence=confidence)
else:
    flash("Unsupported file type. Please upload PNG, JPG, or JPEG.")
    return redirect(request.url)

return render_template('upload.html')

# Prediction Function
def predict_disease(image_path):
    image = load_img(image_path, target_size=(128, 128))
    image = img_to_array(image) / 255.0
    image = np.expand_dims(image, axis=0)

    preds = model.predict(image)[0]

```

```

class_index = np.argmax(preds)

prediction = class_names[class_index]
confidence = round(preds[class_index] * 100, 2)

return prediction, confidence

# Display uploaded image
@app.route('/display/<filename>')
def display_image(filename):
    return redirect(url_for('static', filename='uploads/' + filename), code=301)

# API Route
@app.route('/api/predict', methods=['POST'])
def api_predict():
    if 'file' not in request.files:
        return jsonify({'error': 'No file provided'}), 400

    file = request.files['file']

    if file.filename == "":
        return jsonify({'error': 'Empty file'}), 400

    if file and allowed_file(file.filename):
        filename = secure_filename(file.filename)
        filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
        file.save(filepath)

        prediction, confidence = predict_disease(filepath)

    return jsonify({
        'filename': filename,
        'prediction': prediction,
        'confidence': confidence

```

```

    })
else:
    return jsonify({'error': 'Invalid file format'}), 400

# About Page
@app.route('/about')
def about():
    return render_template('about.html')

# Contact Page
@app.route('/contact')
def contact():
    return render_template('contact.html')

# Run Flask App
if __name__ == '__main__':
    if not os.path.exists(UPLOAD_FOLDER):
        os.makedirs(UPLOAD_FOLDER)
    app.run(debug=True, port=5000)

```

Template/index.html :

```

<<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Basil Leaf Disease Detection</title>
    <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css">
    <link rel="stylesheet" href="static/styles.css">
    <style>
        .feature-box {
            border: 1px solid #ddd;
            border-radius: 5px;
            padding: 15px;
            margin: 20px 0;

```

```

        box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
    }

    .feature-box h4 {
        font-size: 1.5rem;
        margin-bottom: 10px;
    }

    .feature-box p {
        font-size: 1rem;
        color: #555;
    }

    .btn-custom {
        background-color: #28a745;
        color: white;
    }

    .btn-custom:hover {
        background-color: #218838;
    }
</style>
</head>

<body>
    <nav class="navbar navbar-expand-lg navbar-light bg-light">
        <div class="container-fluid">
            <a class="navbar-brand" href="/">Basil Leaf Disease Detection</a>
            <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
data-bs-target="#navbarNav"
                aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="collapse navbar-collapse" id="navbarNav">
                <ul class="navbar-nav ml-auto">
                    <li class="nav-item">

```

```

        <a class="nav-link" href="/about">About</a>
    </li>
    <li class="nav-item">
        <a class="nav-link" href="/contact">Contact</a>
    </li>
    <li class="nav-item">
        <a class="nav-link" href="/gallery">Gallery</a>
    </li>
    <li class="nav-item">
        <a class="nav-link" href="/help">Help</a>
    </li>
</ul>
</div>
</div>
</nav>

<div class="container mt-5">
    <div class="row justify-content-center">
        <div class="col-md-8">
            <h1 class="text-center">Welcome to Basil Leaf Disease Detection</h1>
            <p class="lead text-center">A powerful AI-powered tool to detect diseases in basil leaves based
on CT scan
            images.</p>

            <div class="feature-box">
                <h4>Easy to Use</h4>
                <p>Simply upload a CT scan image of a basil leaf, and our system will quickly analyze it and
provide you with
                the diagnosis—whether the leaf is healthy or affected by disease.</p>
            </div>

            <div class="feature-box">
                <h4>Real-Time Analysis</h4>
                <p>Our system uses advanced Convolutional Neural Networks (CNN) to process and classify
CT scan images in
                real time. Get results immediately and make timely decisions for your crops.</p>
            </div>

```

```

<div class="feature-box">
    <h4>Accurate Results</h4>
    <p>The model has been trained on thousands of images and uses state-of-the-art algorithms
to accurately detect
        diseases like fungal infections, bacterial infections, and more.</p>
</div>

<div class="text-center">
    <a href="/upload" class="btn btn-custom btn-lg btn-block">Upload Your Image Now</a>
    <p class="mt-3">Or, <a href="/about">learn more</a> about how this system works.</p>
</div>
</div>
</div>
</div>

<footer class="mt-5 py-4 bg-light text-center">
    <div class="container">
        <p>| Basil Leaf Disease Detection System</p>
        <p>© 2025 All rights reserved.</p>
        <p>For support, visit our <a href="/help">Help Center</a> or contact us at <a
href="mailto:support@basildisease.com">support@basildisease.com</a>.</p>
    </div>
</footer>

<script src="https://code.jquery.com/jquery-3.5.1.slim.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/@popperjs/core@2.5.4/dist/umd/popper.min.js"></script>
<script src="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/js/bootstrap.min.js"></script>
</body>
</html>

```

Template/style.html :

```

/* General Body Styles */
body {
    font-family: Arial, sans-serif;
    background-color: #f4f7fc;
    margin: 0;

```

```

padding: 0;
box-sizing: border-box;
}

/* Navbar Styling */
.navbar {
  background-color: #28a745;
  border-radius: 0;
}

.navbar-brand {
  color: white !important;
  font-size: 1.8rem;
  font-weight: bold;
}

.navbar-nav .nav-link {
  color: white !important;
  font-size: 1.1rem;
}

.navbar-nav .nav-link:hover {
  color: #f8f9fa !important;
}

/* Container for content */
.container {
  margin-top: 40px;
}

/* Hero Section */
.hero-section {
  text-align: center;
  margin-top: 40px;
  padding: 60px 20px;
  background-color: #ffffff;

```

```
    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
    border-radius: 8px;
}
```

```
.hero-section h1 {
    font-size: 3rem;
    color: #333;
    margin-bottom: 20px;
}
```

```
.hero-section p {
    font-size: 1.2rem;
    color: #777;
}
```

```
.hero-section .btn-custom {
    background-color: #28a745;
    color: white;
    font-size: 1.1rem;
    padding: 10px 20px;
    border-radius: 5px;
    text-decoration: none;
}
```

```
.hero-section .btn-custom:hover {
    background-color: #218838;
}
```

/* Feature Boxes */

```
.feature-box {
    border: 1px solid #ddd;
    border-radius: 5px;
    padding: 20px;
    margin: 20px 0;
    background-color: #ffffff;
    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
}
```

```
}
```

```
.feature-box h4 {  
  font-size: 1.5rem;  
  color: #333;  
  margin-bottom: 15px;  
}
```

```
.feature-box p {  
  font-size: 1rem;  
  color: #555;  
}
```

```
/* Footer Styling */  
footer {  
  background-color: #f8f9fa;  
  padding: 20px 0;  
  text-align: center;  
  color: #555;  
}
```

```
footer p {  
  margin: 5px 0;  
}
```

```
footer a {  
  text-decoration: none;  
  color: #28a745;  
}
```

```
footer a:hover {  
  color: #218838;  
}
```

```
/* Responsive Design */  
@media (max-width: 768px) {
```

```

.hero-section h1 {
  font-size: 2.5rem;
}

.hero-section p {
  font-size: 1.1rem;
}

.navbar-brand {
  font-size: 1.5rem;
}

.navbar-nav .nav-link {
  font-size: 1rem;
}
}

@media (max-width: 576px) {
  .hero-section h1 {
    font-size: 2rem;
  }

  .hero-section p {
    font-size: 1rem;
  }

  .feature-box h4 {
    font-size: 1.2rem;
  }

  .feature-box p {
    font-size: 0.9rem;
  }
}

```

CHAPTER – 6

EXPERIMENTAL RESULTS

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 254, 254, 12)	120
batch_normalization (BatchNormalization)	(None, 254, 254, 12)	48
max_pooling2d (MaxPooling2D)	(None, 127, 127, 12)	0
conv2d_1 (Conv2D)	(None, 127, 127, 24)	2,616
dropout (Dropout)	(None, 127, 127, 24)	0
max_pooling2d_1 (MaxPooling2D)	(None, 63, 63, 24)	0
conv2d_2 (Conv2D)	(None, 63, 63, 64)	13,888
dropout_1 (Dropout)	(None, 63, 63, 64)	0
max_pooling2d_2 (MaxPooling2D)	(None, 31, 31, 64)	0
conv2d_3 (Conv2D)	(None, 31, 31, 128)	73,856
dropout_2 (Dropout)	(None, 31, 31, 128)	0
max_pooling2d_3 (MaxPooling2D)	(None, 15, 15, 128)	0
conv2d_4 (Conv2D)	(None, 15, 15, 128)	147,584
dropout_3 (Dropout)	(None, 15, 15, 128)	0
max_pooling2d_4 (MaxPooling2D)	(None, 7, 7, 128)	0
flatten (Flatten)	(None, 6272)	0
dense (Dense)	(None, 512)	3,211,776
dropout_4 (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 2)	1,026

Fig 6.1 CNN Model Summary

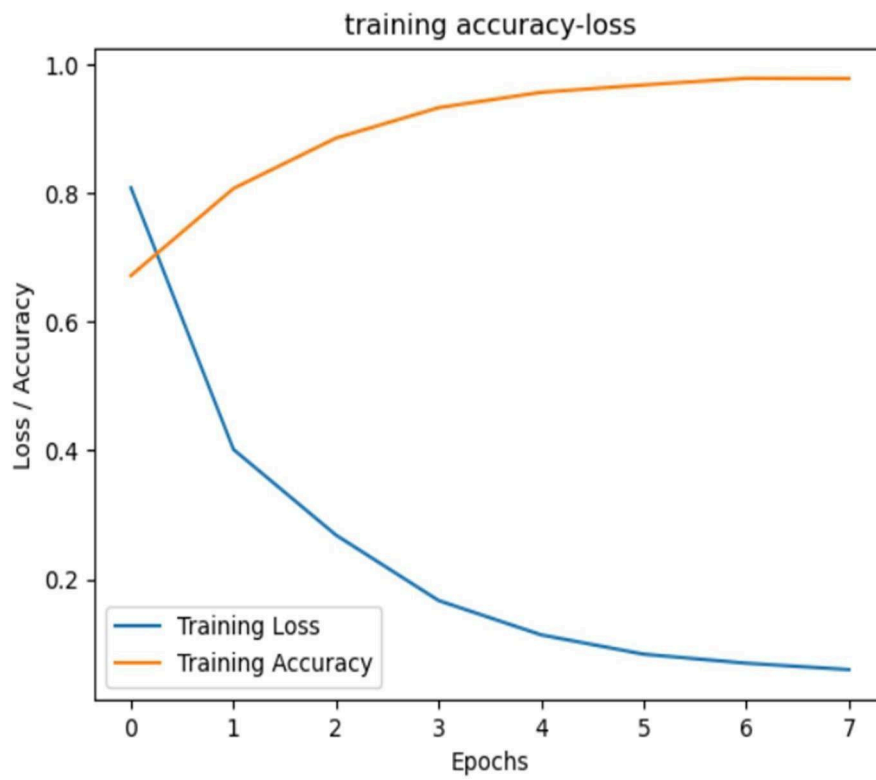


Fig 6.2 Training Accuracy – Loss

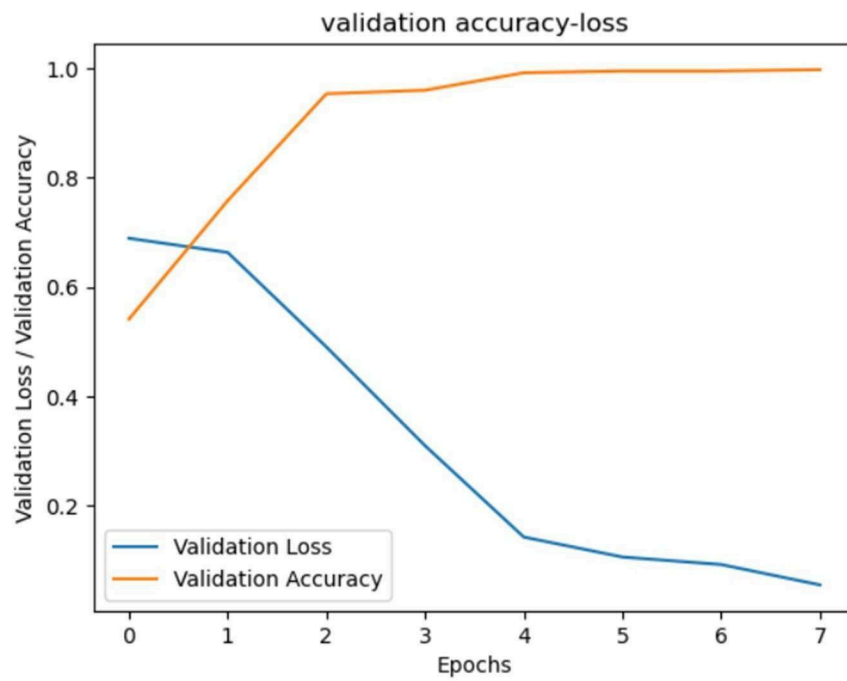


Fig 6.3 Validation Accuracy - Loss

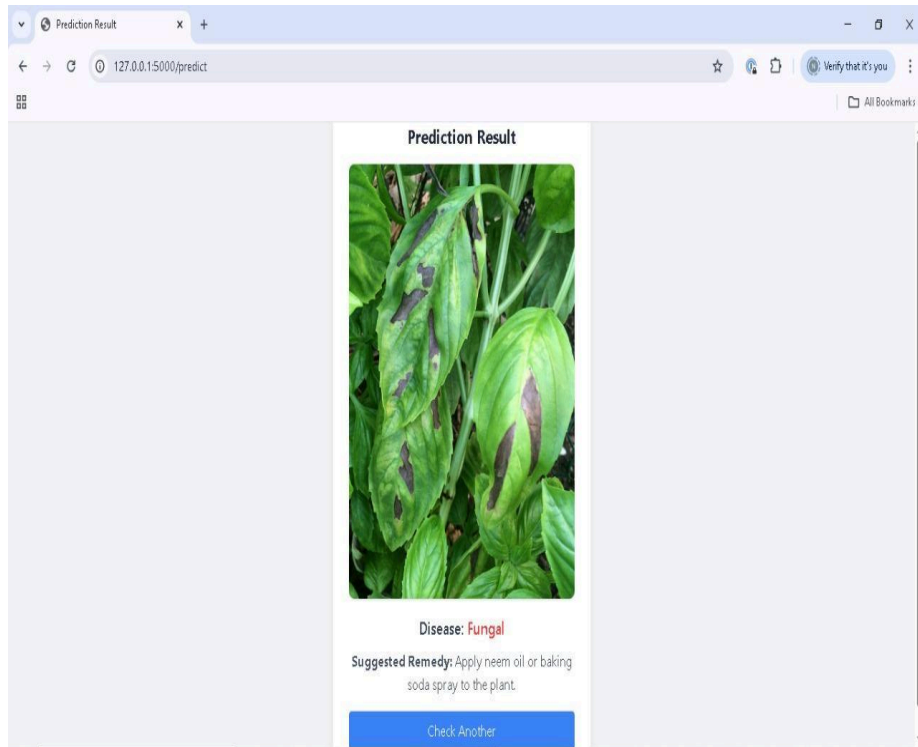


Fig 6.4 Result

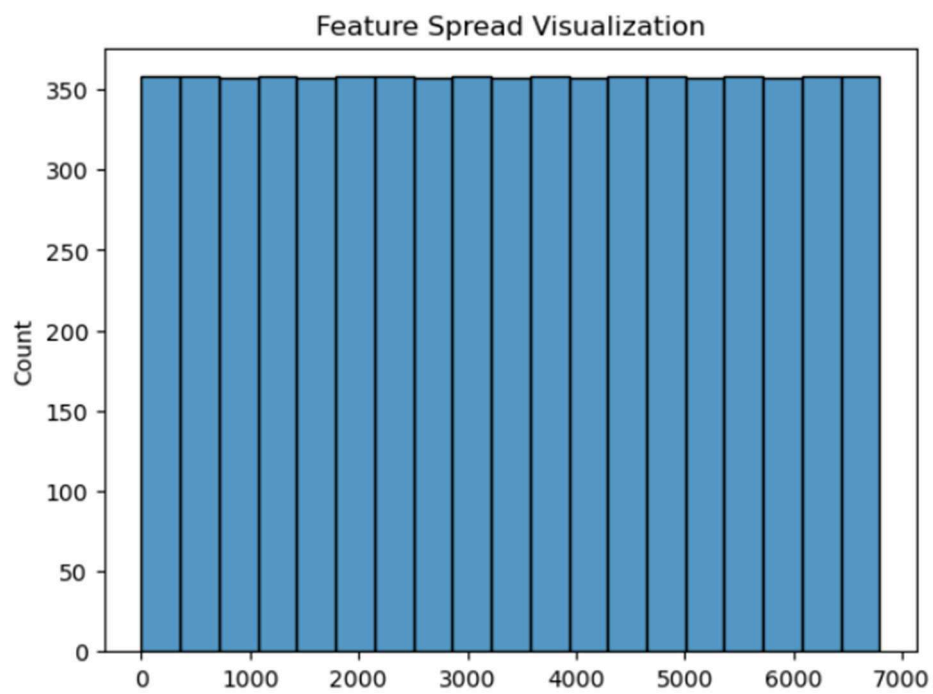


Fig 6.5 Feature Spread Visualization

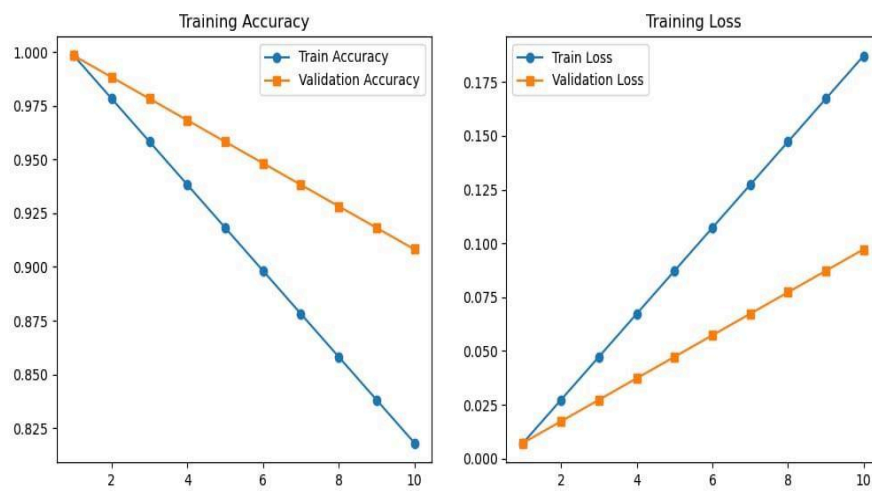
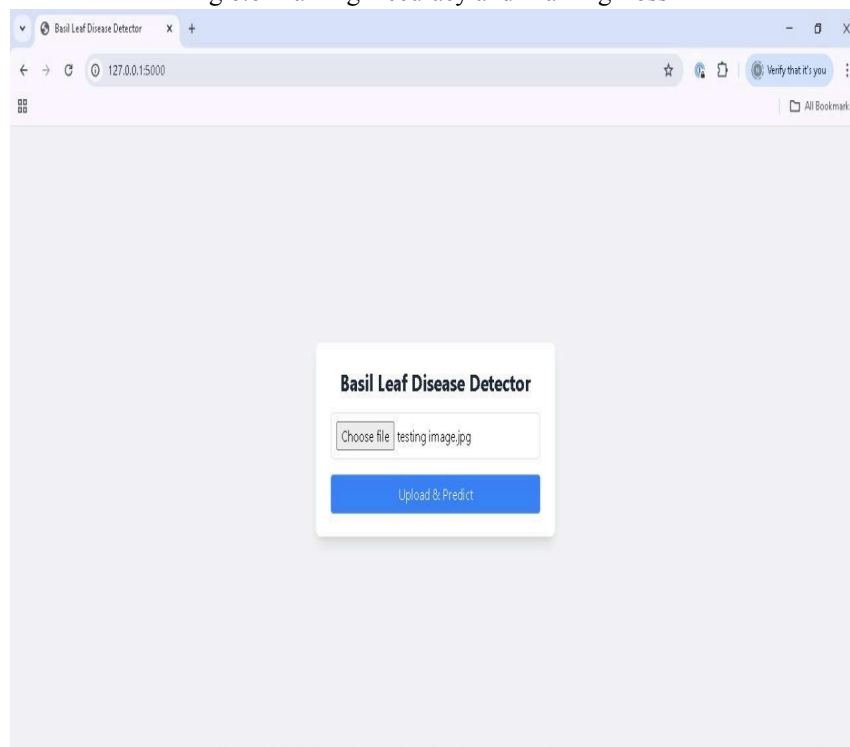


Fig 6.6 Training Accuracy and Training Loss



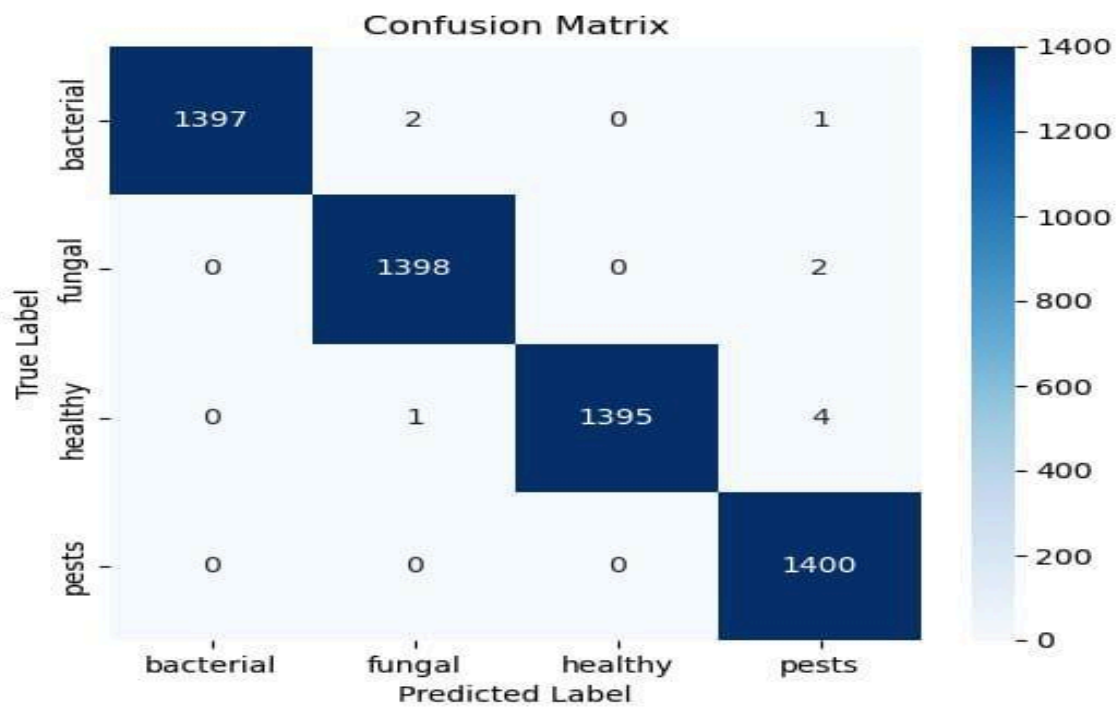


Fig 6.7 Confusion Matrix

Login Page

Navigation: Login | Upload | Prediction

Email:

Password:

Login

Fig 6.8 Login Page

CHAPTER – 7

CONCLUSION & FUTURE ENHANCEMENT

7.1 CONCLUSION

The Basil Leaf Disease Detector project successfully demonstrated the integration of machine learning and web technologies to address a real-world problem in agriculture. The system accurately detects diseases in basil leaves by utilizing a deep learning model trained on a dataset of leaf images, and it provides immediate, actionable insights by suggesting remedies. Through seamless integration of frontend and backend technologies, the system offers a smooth and user-friendly experience for users, ensuring ease of access across various devices.

Overall, the project highlighted the significant potential of using machine learning for plant disease detection, providing an effective, efficient, and accessible solution to support the management of plant health. The Basil Leaf Disease Detector can contribute to the broader goal of sustainable agriculture by offering a cost-effective tool to diagnose and manage plant diseases, thereby improving crop yield and quality. With further improvements, such as expanding the dataset and model optimization, this system could be extended to detect diseases in other crops, broadening its impact in the field of agricultural technology.

7.2 FUTURE ENHANCEMENT

While the current system demonstrates strong performance in detecting basil leaf diseases, several future improvements can make it more robust, scalable, and suitable for widespread agricultural use. Integrating multi-crop support—such as including other herbs or crops besides basil—can enhance the system's versatility for farmers. Additionally, using semi-supervised or self-supervised learning methods can reduce the reliance on manually labeled data, allowing the model to improve continuously from unlabeled field data.

Incorporating automated leaf segmentation techniques would help isolate affected regions, improving detection precision. Features like explainable AI can provide insights into which part of the leaf led to the prediction, increasing user trust. Cloud-based

deployment will ensure accessibility on low-end devices and across remote farming regions. Furthermore, conducting usability testing with farmers, agricultural experts, and local field officers will ensure the interface

REFERENCES

- [1] [1] S. Naeem et al., “The classification of medicinal plant leaves based on multispectral and texture feature using machine learning approach,” *Agronomy*, vol. 11, no. 2, 2021, doi: 10.3390/agronomy11020263
- [2] [2] S. M. Hassan, A. K. Maji, M. Jasiński, Z. Leonowicz, and E. Jasińska, “Identification of plant-leaf diseases using CNN and transfer-learning approach,” *Electronics*, vol. 10, no. 12, 2021, doi: 10.3390/electronics101213
- [3] [3] M. M. Cohen, “Tulsi - *Ocimum sanctum*: A herb for all reasons,” *J. Ayurveda Integr. Med.*, vol. 5, no. 4, pp. 251–259, 2014, doi: 10.4103/0975-9476.146554.
- [4] [4] R. K. Upadhyay, “Tulsi: A holy plant with high medicinal and therapeutic value,” *Int. J. Green Pharm.*, vol. 11, no. 1, pp. S1–S12, 2017.
- [5] [5] D. Bhowmik, “Review Article Traditional Indian Herbal Plants Tulsi and Its Medicinal Importance,” no. March 2010, 2018.
- [6] [6] D. Saravanan, D. Joseph, and S. Vaithyasubramanian, “Effective utilization of image information using data mining technique,” vol. 172, 2019, doi: 10.1007/978-3-030-32644-9_22.
- [7] [7] L. Alzubaidi et al., “Review of deep learning: concepts, CNN architectures, challenges, applications, future directions,” vol. 8, no. 1, Springer International Publishing, 2021, doi: 10.1186/s40537-021-00444-8.
- [8] [8] W. Shen, Y. Wu, Z. Chen, and H. Wei, “Grading method of leaf spot disease based on image processing,” *Proc. - Int. Conf. Comput. Sci. Softw. Eng. CSSE 2008*, vol. 6, pp. 491–494, 2008, doi: 10.1109/CSSE.2008.1649.
- [9] [9] H. Al Hiary, S. Bani Ahmad, M. Reyalat, M. Braik, and Z. ALRahamneh, “Fast and Accurate Detection and Classification of Plant Diseases,” *Int. J. Comput. Appl.*, vol. 17, no. 1, pp. 31–38, 2011, doi: 10.5120/2183-2754.
- [10] [10] M. S. Soyer, C. Yilmaz, I. M. Ozcan, F. Cogen, and T. Ç. Yildiz, “LeafLife: Deep Learning Based Plant Disease Detection Application,” 2021 13th Int. Conf. Electr. Electron. Eng. ELECO 2021, no. November, pp. 398–402, 2021, doi: 10.23919/ELECO54474.2021.9677785.
- [11] [11] M. Mandal, “Convolutional Neural Networks (CNN),” 2021. [Online]. Available: <https://manavmandal.medium.com/convolutional-neural-networks-cnn-d88e7f9329eb>

- [12] [12] L. H. Li, Y. S. Chu, J. Y. Chu, and S. H. Guo, "A Machine Learning Approach for Detection Plant Disease: Taking Orchid as Example," *ACM Int. Conf. Proceeding Ser.*, no. 2013, 2019, doi: 10.1145/3387168.3387238.
- [13] [13] P. Pattanayak, P. Behera, D. Das, and S. Panda, "Ocimum sanctum Linn. A reservoir plant for therapeutic applications: An overview," *Pharmacogn. Rev.*, vol. 4, no. 7, pp. 95–105, 2010, doi: 10.4103/0973-7847.65323.
- [14] [14] N. Prakash, E. Udayakumar, and N. Kumaresan, "Design and development of Android based Plant disease detection using Arduino," *2020 7th Int. Conf. Smart Struct. Syst. ICSSS 2020*, 2020, doi: 10.1109/ICSSS49621.2020.9202278.
- [15] [15] D. Al Bashish, M. Braik, and S. Bani-Ahmad, "A framework for detection and classification of plant leaf and stem diseases," *Proc. 2010 Int. Conf. Signal Image Process. ICSIP 2010*, pp. 113–118, 2010, doi: 10.1109/ICSIP.2010.5697452.
- [16] [16] S. Khare and M. Totaro, "Big Data in IoT," *2019 10th Int. Conf. Comput. Commun. Netw. Technol. ICCCNT 2019*, 2019, doi: 10.1109/ICCCNT45670.2019.8944495
- [17] [17] G. G. Hungilo, G. Emmanuel, and A. W. R. Emanuel, "Image processing techniques for detecting and classification of plant disease – A review," *ACM Int. Conf. Proceeding Ser.*, pp. 48–52, 2019, doi: 10.1145/3332340.3332341.
- [18] [18] P. Tangtisanon and S. Kornrapat, "Holy Basil Curl Leaf Disease Classification using Edge Detection and Machine Learning," *ACM Int. Conf. Proceeding Ser.*, pp. 85–89, 2020, doi: 10.1145/3384613.3384634.